ASLIJOBS ENGINEERING AUDIT

WhatsApp Bot Technical audit

Current architecture, question understanding, accuracy limits, and a prioritized improvement plan. Written from the repository. No application code was changed.

26	3	55	116
runtime components traced	AI paths, two of them off by default	unit tests executed, all passed	proposed cases, not executed

Date. 6 November 2026

Code. `aslijobs/backend/src/modules/whatsapp`, website knowledge, operations inbox

Method. Source reading. Five test files run locally. No live Meta, Sarvam, or OpenAI calls.

Companion files in docs/reports/whatsapp-bot: architecture, intelligence audit, roadmap, and the 116-case evaluation set.

Contents

1. Executive summary	2
2. How a message is handled	3
3. Component inventory	4
4. Understanding and memory	4
5. Website knowledge	5
6. Workflows that exist	5
7. Confirmed limits	6
8. Security and reliability	6
9. Roadmap	6
10. Tests and production checks	7

1. Executive summary

The WhatsApp bot is a backend pipeline on `POST /api/v1/whatsapp/webhook`. It stores the inbound message, claims the Meta message id, and runs one turn per phone. Job vacancies come from the public job query. The model is not the source of job rows.

Question understanding is a local phrase and catalog classifier (`understandLocally`). Sarvam `sarvam-105b` is used only when that classifier is unsure and the text is at least 12 characters. OpenAI rephrasing and multilingual E5 retrieval exist in code and default to off.

The bot can continue a search across short follow-ups such as a city, a salary, or “show more”. It cannot compare pay across the jobs it just showed. “Yes” and “No” are acknowledgements and do not answer an open clarification. A greeting clears the active search.

Website answers are copied from retrieved pages, with a source URL. They are not written by a chat model. With the default hash embedder, a Telugu question will not semantically match an English page.

Production readiness depends on environment facts this snapshot cannot see: Meta secrets, Redis, the Sarvam key, approved template names, and whether semantic retrieval is actually enabled on the host.

2. How a message is handled

```
Meta  POST /api/v1/whatsapp/webhook
  verifyWhatsAppSignature          whatsapp-signature.ts
  recordInboundUserMessage         whatsapp-inbox-store.ts
  HTTP 200
  claimWhatsAppEvent               whatsapp-processed-event.model.ts
  pause? human inbox only
  enqueueWhatsAppInbound           BullMQ "whatsapp-inbound" or inline
  runInPhoneOrder
  processInboundTurn               whatsapp-bot.service.ts
     account lookup + WhatsAppSession
     protocol (hi / yes / no) OR understandLocally
     optional Sarvam JSON classify
     resolveTurnUnderstanding + role gate
     buildReply (jobs, applications, or knowledge)
     localizeDeterministicReply
     optional OpenAI rephrase
     sendBotTextMessage
```

OTP uses `WhatsAppService.sendOtpMessage`. That writes an inbox conversation through `recordOutboundMessage` → `ensureInboxConversation`. It does not create a `WhatsAppSession`. Bot memory starts on the first inbound turn.

Redis is the BullMQ transport. Conversation slots live in Mongo, collection via `WhatsAppSessionModel`, and expire 14 days after `lastInteractionAt`.

3. Component inventory

Component	Responsibility	Primary file
Webhook verify and receive	Subscribe challenge and signed POST	whatsapp-webhook.controller.ts
Signature	HMAC SHA-256 of the raw body	whatsapp-signature.ts
Inbound queue	BullMQ, 2 attempts, else inline	whatsapp-inbound.queue.ts
Phone lock	One turn at a time per phone	whatsapp-phone-order.ts
Session	Language, search slots, last jobs	whatsapp-session.model.ts
Local NLU	Intents, places, roles, follow-ups	whatsapp-bot.logic.ts
Criteria	Salary, experience, ordinals	whatsapp-bot.criteria.ts
Sarvam chat	JSON intent fallback, 8s timeout	sarvam.client.ts
Sarvam STT	Voice notes, two model attempts	sarvam.client.ts
Sarvam translate	Localize deterministic replies	sarvam-translate.client.ts
OpenAI assist	Rephrase only, default off	openai-assist.client.ts
Role gate	Seeker and employer isolation	whatsapp-role-policy.ts
Tools	Three allowlisted queries	whatsapp-bot-tools.ts
Orchestrator	No-match menu 1–4	whatsapp-conversation-orchestrator.ts
Cloud API	Text, templates, media download	whatsapp.service.ts
OTP	Authentication template	whatsapp-otp.provider.ts
Notifications	Account, job, interview templates	notifications/*.ts
Inbox	Persist, pause bot, delivery status	inbox/whatsapp-inbox-store.ts
Operations UI	Staff inbox and translation view	admin inbox components
Knowledge crawl	Public pages, chunks, FAQ	website-knowledge.sync.ts
Embeddings	Hash 256-d default, E5 384-d optional	website-knowledge.embeddings.ts
Knowledge answer	Extract plus URL, no LLM	website-knowledge.answer.ts
Media	Resume OCR, flag off by default	whatsapp-media-understand.service.ts
Rate limit	20 replies per phone per minute, in-process	allowReply

4. Understanding and memory

Intents are the `BOT_INTENTS` list: greeting, job search, job details, counts, profile, applications, employer jobs, post job, account status, platform knowledge, help, clarify, unrelated, unknown.

Sarvam's system prompt asks for JSON only and forbids inventing jobs. The user payload is the current text (280 characters), the account label, and previous city and role only when `isFollowUpFragment` is true (at most 48 characters and a bare place or role).

Language is detected on the original text. The bot does not translate into English before classification. Romanized cues are finite word lists (`ROMAN_TELUGU` , `ROMAN_HINDI` , and the Tamil, Kannada, and Malayalam lists). Unit tests for those detectors passed. The lists are not a complete model of romanized speech.

There is no stored chat transcript on the session. Follow-ups work from slots: `lastCategory` , `lastLocation` , salary, experience, and `lastJobs` . Tests that passed show React kept when the next message adds Hyderabad,

salary added without dropping driver and city, and show-more keeping the same search.

> "Which one pays more?" is not handled. `looksLikeRelativeHigherSalary` only matches phrases such as "higher salary" and "konchem ekkuva", then adds 5000 to the previous minimum and searches again. It does not rank `lastJobs`.

5. Website knowledge

Crawl and chunk code writes FAQ and section documents. Default embedding is `aslijobs-hash-v1` (256 dimensions, hashed tokens, L2-normalized). Optional model is `Xenova/multilingual-e5-small` (384 dimensions). `embeddingsCompatible` refuses to mix them. If E5 fails to load, inference falls back to hash.

Dense search runs only when `WEBSITE_KNOWLEDGE_SEMANTIC_ENABLED` is true. Cosine under 0.22 is ignored. The answer function pastes sanitized chunk text and a canonical URL. A miss returns a fixed line plus Help Center, Contact, and FAQs. Retrieved lines that look like prompt injection are stripped.

The bot calls this only for `PLATFORM_KNOWLEDGE`. Live jobs never come from this index. OpenAI is not allowed to rephrase knowledge answers.

6. Workflows that exist

Job seeker

Implemented in the turn handler: public search, counts, details for a numbered result, show more, profile-based search, application list and status, how-to-apply as a link, account status, voice notes when Sarvam STT is configured. Not implemented: creating an application inside WhatsApp. Resume OCR exists and is off unless `WHATSAPP_MEDIA_UNDERSTAND_ENABLED` is true. It stores skills on the session for 24 hours and does not write the profile.

Employer

Implemented: verification copy, post-job link (no in-chat draft), own job list, application counts. Not implemented in the bot: interview scheduling, draft delete, or approval. Those events are outbound templates in `whatsapp-notification.constants.ts` (account created, profile reminders, job submitted/approved/rejected/incomplete, application received, interview scheduled/rescheduled/cancelled).

The bot sends session text. Interactive lists are not sent. Inbound button and list titles are flattened to text in `extractText`.

7. Confirmed limits

Limit	Why it happens	Where
Yes/No ignore an open clarify	Protocol ack returns before slot merge	detectProtocolTurn
Hello clears the search	Greeting spreads clearedTemporarySearchSessionFields	processInboundTurn
Short text skips Sarvam	Length under 12 never calls the LLM	shouldUseLlmFallback
No pay comparison on the shown list	Relative salary only adds 5000 to the last minimum	mergePending
One intent per message	BotUnderstanding has a single intent	whatsapp-bot.logic.ts
Indic question vs English page	Default embedder hashes the query's own tokens	embedKnowledgeTextHash
Extra "looking at your question" bubble	Sent in parallel with every Sarvam call	processInboundTurn
Silent rate limit	21st reply in a minute returns with no send	allowReply
Ack before routing finishes	HTTP 200 after inbox persist, then route	receiveWhatsAppWebhook

These are source findings. This audit did not count how often they occur in production.

8. Security and reliability

- Verify token compare is timing-safe. The hub challenge must match a short token pattern.
- HMAC is required when a secret is required. Production startup refuses unsigned webhooks.
- Webhook limiter is 600 requests per minute. The path is skipped by the global API limiter.
- Verify token is redacted in access logs.
- Non-Indian numbers are not mapped onto Indian accounts. Tests passed.
- Blocked or suspended accounts are not linked. Tests passed.
- Employer and seeker tools are gated before queries. The tool list is search public jobs, list seeker applications, list employer jobs.
- OpenAI text that claims an application was submitted is rejected. Test passed.
- Inbound worker: 2 attempts, failed jobs kept (100). No separate dead-letter queue in the inbound file.
- Without Redis, turns and notification sends run in-process and do not survive a restart. `env.ts` logs that warning.

9. Roadmap

P0. Honor yes/no while a clarification is pending. Send one line when the rate limit trips, and move the counter to Redis. Acknowledge the webhook after enqueue succeeds.

P1. Compare salary labels on `lastJobs` for "which pays more". Tell the user when shift is not a filter. Give "jobs" and "apply" a local intent instead of skipping the classifier because they are short. Reindex knowledge with E5 only as a matched 384-dimension index. Shorten knowledge extracts. Do not add an answer LLM.

P2. Send last slots to Sarvam on every non-protocol turn. Split a message only when two known intents are both obvious. Decide whether "Hi" should keep the search; the reset is intentional in the current comment.

P3. Delay the classifier acknowledgement. Keep the existing masked `[WA-FLOW]` logs.

Not recommended now. Replacing Sarvam as the primary brain, letting a model query Mongo, or submitting applications inside WhatsApp.

10. Tests and production checks

Executed on 6 November 2026, 55 passed, 0 failed:

- whatsapp-language-detection.test.ts
- whatsapp-conversation-context.test.ts
- whatsapp-security.test.ts
- whatsapp-bot.criteria.test.ts
- website-knowledge.e2e-closure.test.ts

The repository's full `npm test` script was not run. The 116-row evaluation set in `WHATSAPP_BOT_TEST_CASES.md` was not executed. It covers English, Telugu, romanized Telugu, Hindi, Tamil, Kannada, Malayalam, mixed language, spelling, follow-ups, employer actions, knowledge, injection, and media flags.

Before calling the bot production-ready, operations still need to confirm on the running host: `WHATSAPP_VERIFY_TOKEN`, `META_APP_SECRET`, `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `REDIS_URL`, `SARVAM_API_KEY`, template names versus Meta Manager, and whether `WEBSITE_KNOWLEDGE_SEMANTIC_ENABLED` and the embedding provider match the stored chunks. Those values were not read from production.

End of report. Detailed write-ups are the four Markdown files in the same folder.